

E-COMMERCE APP FOR ARM STEEL AGENCY

A PROJECT REPORT (22ITP61-Project Work I)

Submitted by

**Name of the student
(23ITRxxx)**

**Name of the student
(23ITRxxx)**

*in partial fulfilment of the requirements
for the award of the degree
of*

BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY



PERUNDURAI ERODE – 638 060

MARCH 2026

DEPARTMENT OF INFORMATION TECHNOLOGY

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638060

MARCH 2026

BONAFIDE CERTIFICATE

This is to certify that the Project report entitled **“E-COMMERCE APP FOR ARM STEEL AGENCY”** is the Bonafide record of project work done by **Name of the student (23ITRxxx)** and **Name of the student (23ITRxxx)** in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in **INFORMATION TECHNOLOGY** of Anna University Chennai during the year 2025 - 2026.

SUPERVISOR

Date:

HEAD OF THE DEPARTMENT

(Signature with seal)

Submitted for the end semester viva voce examination held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DEPARTMENT OF INFORMATION TECHNOLOGY

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638060

MARCH 2026

DECLARATION

We affirm that the Project Report titled “ **E-COMMERCE APP FOR ARM STEEL AGENCY** ” being submitted in partial fulfilment of the requirements for the award of Bachelor of Technology is the original work carried out by us. It has not formed the part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

**Name of the student
(23ITRxxx)**

**Name of the student
(23ITRxxx)**

Date:

I certify that the declaration made by the above candidates is true to the best of my knowledge.

Date:

Name and Signature of the Supervisor with seal

ACKNOWLEDGEMENT

First and foremost, we acknowledge the abundant grace and presence of Almighty throughout different phases of the project and its successful completion.

We express our gratefulness to Correspondent **THIRU. E. R. K. Krishnan, M.Com.** and all the trust members of Kongu Vellalar Institute of Technology Trust for providing all the necessary facilities to complete the project successfully.

We express our deep sense of gratitude to our beloved Principal **Dr.R.Parameshwaran M.E.,Ph.D.**, for providing us an opportunity to complete the project.

We express our gratitude to **Dr. S. Anandamurugan M.E.,Ph.D.**, Head of the Department, Department of Information Technology for his valuable suggestions.

We are thankful to our Project Coordinators **Dr.E.M.Roopu Devi M.E,Ph.D.**, **Ms.S.Sujitha M.Tech** and **G.Sasikala M.E.**, for their valuable guidance and support.

We are highly indebted to **Dr. T. ABIRAMI MCA, ME, PhD.**, Department of Information Technology for her valuable supervision and advice for the fruitful completion of the project.

We are thankful to the faculty members of the Department of Information Technology for their valuable guidance and support.

We are obliged to thank **Mr. xxxx**, Owner of (**Company Name**) for supporting us through the various development phases of the project with utmost trust.

ABSTRACT

An e-commerce mobile application **for ARM Steel Agency** to revolutionize their steel retail operations through digital transformation. The application provides a complete online shopping experience, enabling customers to browse products, place orders, and make secure payments while giving the business robust tools for inventory and sales management. By implementing this digital solution, ARM Steel Agency has bridged the gap between traditional steel retail and modern e-commerce practices.

A standout feature of this system is its seamless Razorpay payment integration, which ensures fast, reliable, and secure financial transactions. The application also includes a comprehensive admin dashboard that gives business owners real-time visibility into inventory levels, order status, and customer activity. This dual-focused approach benefits both customers, who enjoy convenient 24/7 access to steel products, and the business, which gains operational efficiency and valuable sales insights.

Beyond its current capabilities, the application is built on a scalable architecture designed to accommodate future enhancements. Planned expansions multi-branch inventory synchronization, and customer loyalty programs. These upcoming features will further elevate the platform's value proposition, making it an increasingly powerful tool for business growth.

Ultimately, this e-commerce solution has positioned ARM Steel Agency as a forward-thinking player in the steel retail sector. By combining user-friendly design with robust functionality, the application delivers immediate business benefits while establishing a foundation for long-term digital success. The project demonstrates how traditional industries can leverage mobile technology to improve customer experiences, streamline operations, and drive sustainable growth in an increasingly digital marketplace

TABLE OF CONTENTS

CHAPTER No.	TITLE	PAGE No.
	ABSTRACT	iv
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	ix
1	INTRODUCTION	1
	1.1 OVERVIEW	1
	1.2 OBJECTIVE	1
2	SYSTEM SPECIFICATION	2
	2.1 HARDWARE REQUIREMENTS	2
	2.2 SOFTWARE REQUIREMENTS	2
	2.3 SOFTWARE DESCRIPTION	3
	2.3.1 Visual Studio Code	3
	2.3.2 NodeJS	3
	2.3.3 MongoDB	5
	2.3.4 React Native	5
3	SYSTEM DESIGN	8
	3.1 USE CASE DIAGRAM	8
	3.2 CLASS DIAGRAM	9
	3.3 SEQUENCE DIAGRAM	10
	3.4 ACTIVITY DIAGRAM	11
	3.5 DATABASE DESIGN	12

	3.6 MODULES DESCRIPTION	14
	3.6.1 User Authentication Module	14
	3.6.2 Product Management Module	14
	3.6.3 Product Browsing and Search Module	14
	3.6.4 Cart and Order Placement Module	15
	3.6.5 Payment Integration Module	15
	3.6.6 Admin Dashboard Module	15
4	TESTING	16
	4.1 SYSTEM TESTING	16
	4.2 UNIT TESTING	16
	4.3 MODULE TESTING	17
	4.4 INTEGRATION TESTING	17
	4.5 VALIDATION TESTING	17
5	RESULTS & DISCUSSIONS	19
6	CONCLUSION	20
	APPENDIX 1- CODING	21
	APPENDIX 2- SNAPSHOTS	43
	REFERENCES	52

LIST OF FIGURES

FIGURE No.	FIGURE NAME	PAGE No.
3.1	USE CASE DIAGRAM	8
3.2	CLASS DIAGRAM	9
3.3	SEQUENCE DIAGRAM	10
3.4	ACTIVITY DIAGRAM	11
A2.1	SIGNUP SCREEN	43
A2.2	LOGIN SCREEN	43
A2.3	LOGIN INPUT SCREEN	44
A2.4	HOME SCREEN	44
A2.5	HOME SCREEN - SCROLLED	45
A2.6	WISHLIST SCREEN	45
A2.7	ADDING ITEM TO WISHLIST	46
A2.8	REMOVING ITEM FROM WISHLIST	46
A2.9	PROFILE SCREEN	47
A2.10	CART SCREEN	47
A2.11	CHECKOUT SCREEN	48
A2.12	CHECKOUT DETAILS	48
A2.13	INITIALIZING PAYMENT	49
A2.14	PAYMENT SCREEN	49
A2.15	PAYMENT CONFIRMATION	50
A2.16	FILTER BASED ON CATEGORIES	50
A2.17	CATEGORIES SCREEN	51
A2.18	SEARCH PRODUCTS	51

LIST OF ABBREVIATIONS

CSS	-	Cascading Style Sheets
HTML	-	HyperText Markup Language
ID	-	Identification
JS	-	JavaScript
JSX	-	JavaScript XML
QR	-	Quick Response
URL	-	Uniform Resource Locator
AI	-	Artificial Intelligence
API	-	Application Programming Interface
HTTP	-	HyperText Transfer Protocol
AJAX	-	Asynchronous JavaScript and XML
I/O	-	Input/Output.
JSON	-	JavaScript Object Notation.
JWT	-	JSON Web Token.

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

The growing demand for digital transformation in industrial sectors has made it essential for businesses to adopt efficient and scalable solutions for managing operations. This project focuses on developing a mobile-based e-commerce application specifically for ARM Steel Agency, aiming to streamline product sales, inventory tracking, and customer engagement. The application enables users to browse steel products, place orders, and make secure payments through an integrated gateway, while also allowing administrators to manage stock and monitor transactions in real time. By automating key business processes, this solution reduces manual workload, minimizes errors, and enhances operational efficiency. It lays the groundwork for long-term scalability, improved customer service, and future integration with advanced business systems.

1.2 OBJECTIVE

- To develop a comprehensive e-commerce mobile application for ARM Steel Agency to streamline product sales, inventory management, and customer engagement.
- To integrate real-time product updates, secure payment gateway (via Razorpay), and an intuitive user interface to enhance the shopping experience.
- To optimize business operations by automating transactions and providing an efficient, secure platform for customers and admins.

CHAPTER 2

SYSTEM SPECIFICATION

2.1 HARDWARE REQUIREMENTS

Processor	:	Intel Pentium
Processor Speed	:	250MHz to 667MHz
RAM	:	4GB RAM
Hard Disk	:	256GB
Keyboard	:	Standard 104 enhanced
Mouse	:	Local PS/2

2.2 SOFTWARE REQUIREMENTS

Platform	:	Visual Studio Code
Server-Side Script	:	NodeJS
Operating System	:	Windows, macOS, or Linux
Backend	:	MongoDB
Library	:	React Native

2.3 SOFTWARE DESCRIPTION

2.3.1 Visual Studio Code

Visual Studio Code is a versatile and user-friendly code editor used by developers worldwide. It supports a wide range of programming languages and frameworks, including JavaScript, React, and Node.js. For this project, VS Code provides a convenient environment for writing, editing, and debugging code related to server-side scripting with Node.js, as well as client-side scripting with React.js. Its built-in terminal and debugging tools streamline the development process, while its customizable features allow developers to tailor the editor to their specific needs, enhancing productivity and efficiency throughout the project lifecycle.

- **ES6+ React/Redux/React-Native:**

ES6 React/Redux/React-Native snippets provide code templates designed to simplify and accelerate the development process for building applications with React, Redux, and React Native. These snippets utilize the powerful features introduced in ES6, allowing developers to quickly scaffold common patterns like React components, Redux actions, reducers, and React Native components. By incorporating ES6 syntax, such as arrow functions, destructuring, template literals, and modules, these snippets make it easier to write clean and concise code. With ES6 snippets, developers can efficiently generate standardized code structures that promote consistency and reduce the chance of errors, cutting down on boilerplate code. This collection of snippets supports the creation of both web applications in React and cross-platform mobile applications in React Native, while simplifying state management in Redux. Whether developing user interfaces, managing state, or working on cross-platform features, these ES6 snippets help developers stay focused on creating features and solving problems instead of repeating boilerplate setup.

2.3.2 NodeJS

Node.js is a powerful server-side JavaScript runtime environment commonly used for building scalable and efficient web applications. In our project, Node.js serves as

the server- side scripting language, handling tasks such as routing, handling HTTP requests, and interacting with the MongoDB database. Its event-driven architecture and non- blocking I/O operations make it well-suited for handling concurrent connections and processing requests efficiently. With its vast ecosystem of libraries and frameworks, Node.js allows for rapid development and deployment of server-side applications. Additionally, its compatibility with Visual Studio Code provides a seamless development experience, enabling developers to write, test, and debug server-side code effectively within a unified environment. Overall, Node.js plays a crucial role in this project by powering the backend infrastructure and facilitating the seamless integration of frontend and backend components.

- **Bcryptjs**

Bcryptjs is a library used for hashing passwords in JavaScript. It provides a secure way to hash passwords before storing them in a database or comparing them during authentication processes. By using bcryptjs, developers can ensure that user passwords are encrypted and protected against common security threats like brute force attacks and rainbow table attacks.

- **Express**

Express is a minimalist web application framework for Node.js. It provides a robust set of features for building web servers and APIs, including routing, middleware support, template engines, and HTTP utilities. With its simple and flexible design, Express enables developers to quickly create scalable and maintainable web applications

- **Jsonwebtoken**

JSON Web Token is an open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object. JWTs are commonly used for authentication and authorization in web applications. The jsonwebtoken library provides utilities for generating, parsing, and verifying JWTs, making it easy to implement token-based authentication and authorization systems.

2.3.3 MongoDB

MongoDB is a flexible and scalable NoSQL database platform ideal for managing complex data structures, making it invaluable for our project's e-commerce website. With MongoDB's document-oriented architecture, we can efficiently store and manage diverse types of data relevant to our e-commerce platform, such as user information, appointment details, and messages. Its schema-less nature allows for dynamic and evolving data models, accommodating the dynamic nature of e-commerce data. For this project, MongoDB's scalability is crucial as it can seamlessly handle increasing volumes of data and traffic, ensuring optimal performance even during peak usage periods. This scalability ensures that the website remains responsive and can accommodate growth without sacrificing performance. MongoDB's powerful query language and indexing capabilities enable us to perform complex queries efficiently, facilitating tasks such as retrieving user information, appointment details, and message history. This enhances the overall user experience by providing fast and relevant data retrieval. Additionally, MongoDB's support for high availability and automatic failover ensures that our e-commerce website remains accessible and reliable. This feature minimizes downtime and ensures uninterrupted service for users, which is essential for maintaining customer satisfaction and trust. Overall, MongoDB provides a robust and scalable e-database solution that empowers us to create a secure, efficient, and highly performant e-commerce website. Its flexibility, scalability, and powerful features make it an indispensable component of our project, enabling us to meet the diverse data management needs of our e-commerce platform effectively.

2.3.4 React Native

React Native, a powerful framework built on top of React, is central to the development of our e-commerce mobile application. Its component-based architecture enables the creation of reusable, modular UI elements tailored for both Android and iOS platforms. By leveraging native components and a single JavaScript codebase, React Native ensures high performance and consistent user experience across devices. The use of

a virtual DOM-like architecture allows for efficient rendering, while hot-reloading speeds up development and debugging. Additionally, its rich ecosystem and strong community support provide access to numerous third-party libraries and tools, enabling rapid development and seamless integration with backend services like Node.js and MongoDB. React Native's flexibility and cross-platform capability make it an ideal choice for scalable and efficient mobile app development in our e-commerce project.

- **axios:**

Axios is a promise-based HTTP client for making HTTP requests in browser-based and Node.js applications. It provides a simple and intuitive API for performing asynchronous HTTP requests, handling request and response data, and intercepting request and response lifecycle events. Axios supports features like request and response interception, request cancellation, automatic JSON data parsing, and more, making it a popular choice for handling AJAX requests in JavaScript applications.

- **react-navigation:**

react-navigation is a popular routing and navigation library specifically designed for React Native applications. It enables smooth and intuitive navigation between screens using stacks, tabs, and drawer navigators. The library handles navigation history, parameter passing, and screen transitions efficiently across both Android and iOS platforms. It provides a customizable and scalable solution for managing complex navigation flows in mobile apps, ensuring a seamless user experience and structured app architecture.

- **@react-native-async-storage/async-storage:**

AsyncStorage is a local key-value storage system used to persist data on the device. It is helpful for storing user sessions, preferences, or app data that needs to be retained across app restarts. It provides a simple, asynchronous API for reading and writing data securely and efficiently in React Native.

- **razorpay-react-native:**

razorpay-react-native is a library that enables seamless integration of Razorpay's payment gateway into React Native applications. It provides a secure and user-friendly interface for handling online transactions, supporting multiple payment methods like UPI, cards, net banking, and wallets. This library allows developers to initiate and manage payment flows directly within the mobile app, ensuring fast, reliable, and secure payments for e-commerce applications.

- **react-native-vector-icons:**

This library provides a wide range of customizable icons for use in React Native apps. It supports popular icon sets such as FontAwesome, MaterialIcons, and Ionicons. Developers can easily add scalable vector icons to buttons, tabs, headers, and other UI components, enhancing the visual appeal and usability of the application.

CHAPTER 3

SYSTEM DESIGN

3.1 USE CASE DESIGN

A use case diagram is a dynamic or behavior diagram in UML (Unified Modelling Language). Use case diagrams model the functionality of a system using actors and use cases. Use cases are a set of actions and functions that the system needs to perform. A "system" is something being developed or operated, here is a website. The "actors" are people or entities operating under defined roles within the system as shown below in Figure 3.1.

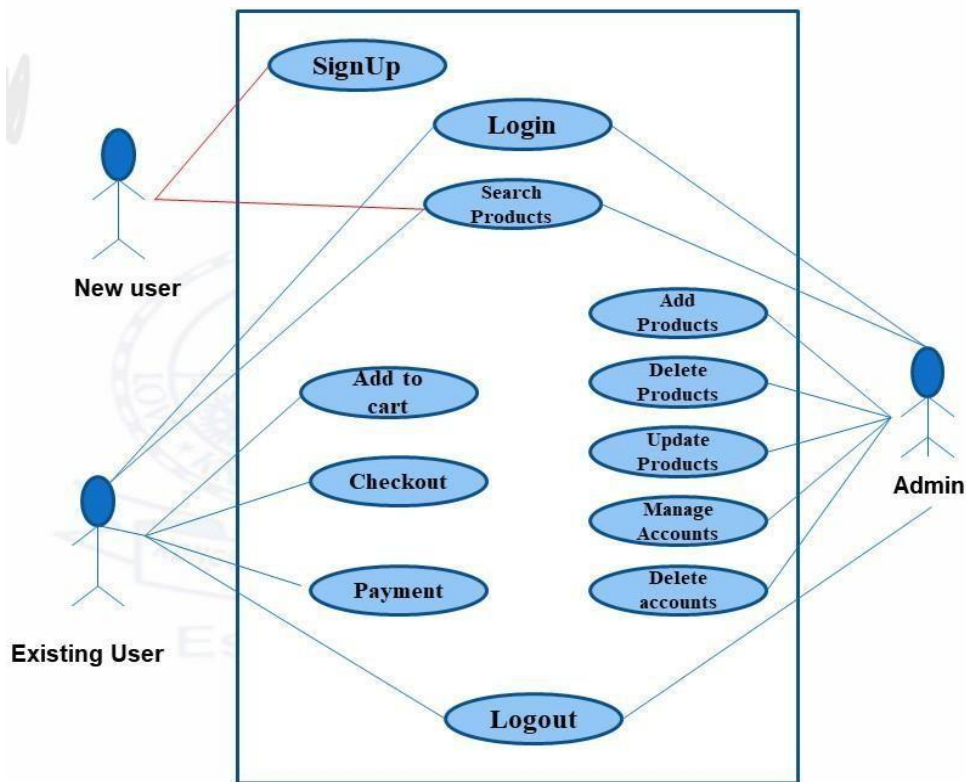


Fig 3.1 Use Case Diagram

3.2 CLASS DIAGRAM

A class diagram is a static structure diagram in UML (Unified Modeling Language) that illustrates the structure of a system by showing the classes of the system, their attributes, methods, and relationships between them as shown in Figure 3.2. Classes represent the blueprint for objects, encapsulating data and behavior. Attributes are the data members of a class, while methods represent the operations that can be performed on the class's objects.

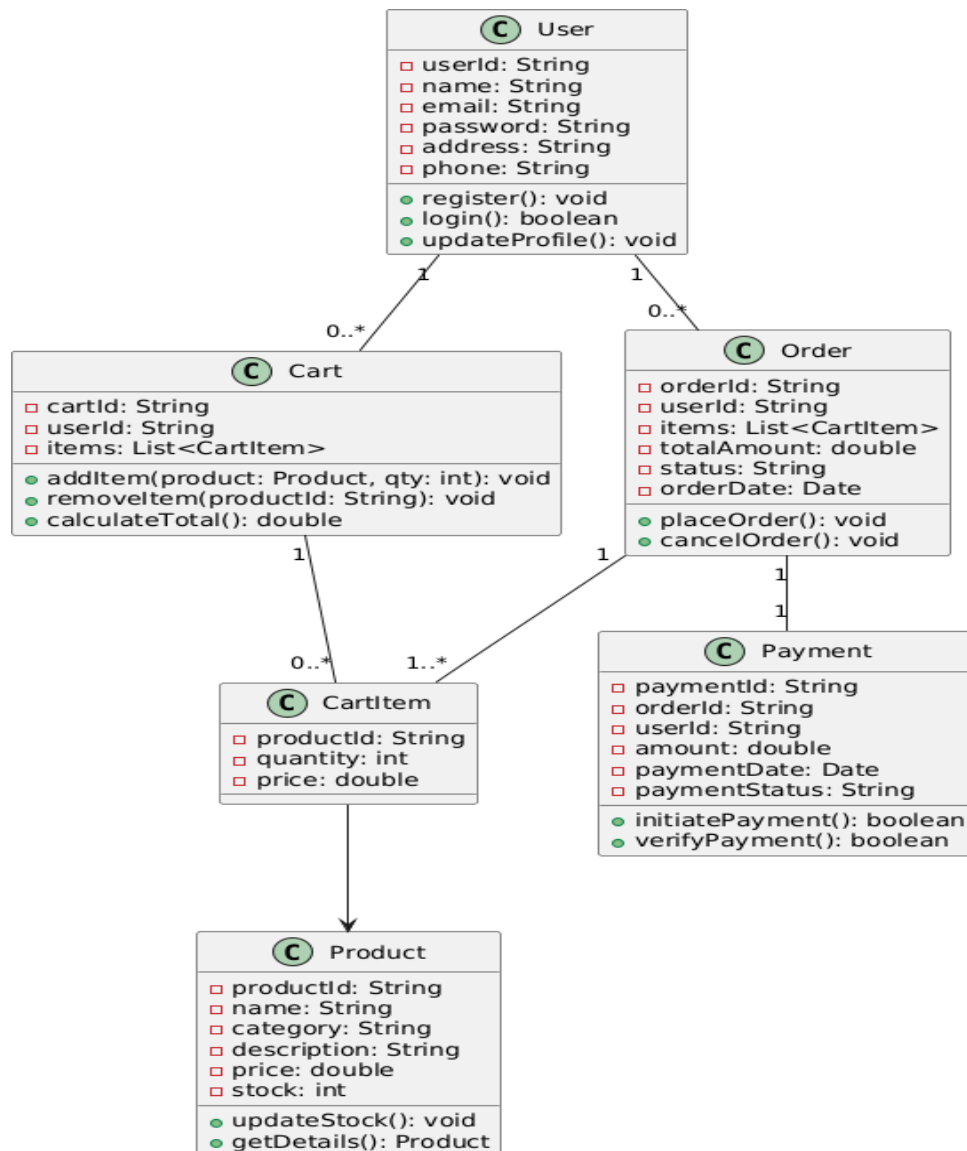


Fig 3.2 Class Diagram

3.3 SEQUENCE DIAGRAM

A sequence diagram is a form of interaction diagram which shows objects as lifelines running down the page, with their interactions over time represented as messages drawn as arrows from the source lifeline to the target lifeline. Sequence diagrams are good at showing which objects communicate with which other objects and what messages trigger those communications as shown below in Figure 3.3. Sequence diagrams are not intended for showing complex procedural logic.

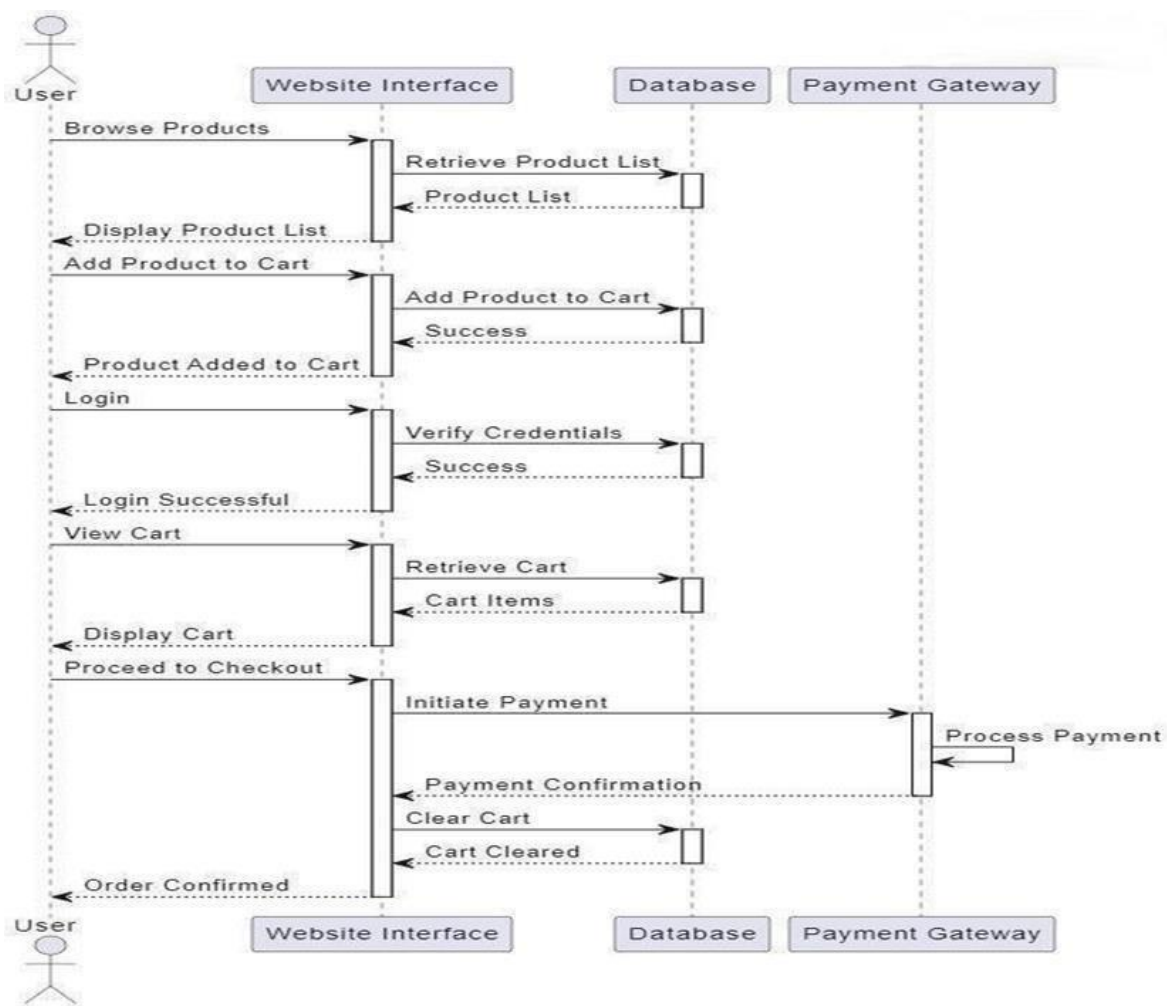


Fig 3.3 Sequence Diagram

3.4. ACTIVITY DIAGRAM

In UML, the activity diagram is used to demonstrate the flow of control within the system rather than the implementation. It models the concurrent and sequential activities. The activity diagram helps in envisioning the workflow from one activity to another. It put emphasis on the condition of flow and the order in which it occurs. The flow can be sequential, branched or concurrent and to deal with such kinds of flows, the activity diagram has come up with a fork, join, etc. as shown in the Figure 3.4 below.

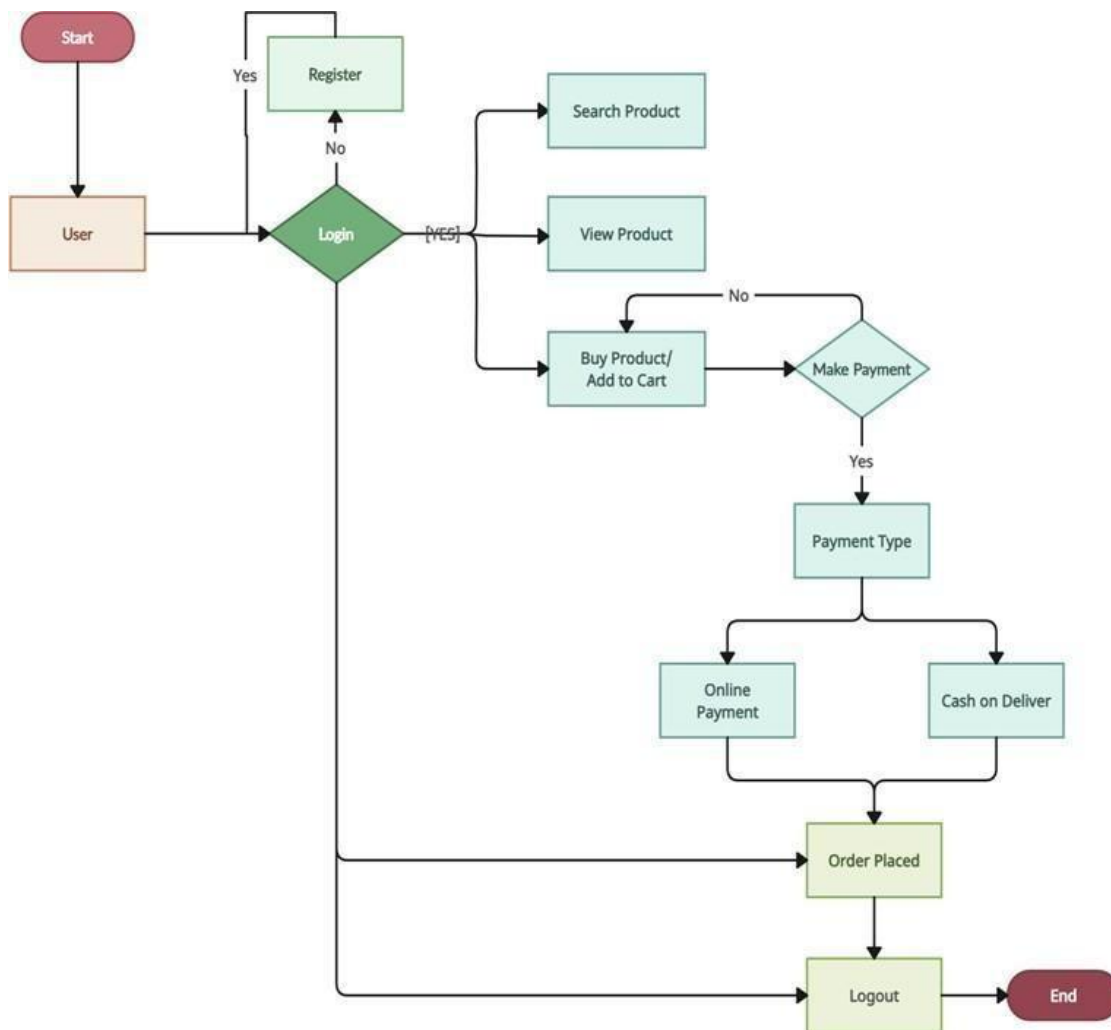


Fig 3.4 Activity Diagram

3.5 DATABASE DESIGN

Product Schema

```
const ProductSchema = new mongoose.Schema({  
  
  name: String,  
  
  Dimensions: String,  
  
  Brand: String,  
  
  quantity: Number,  
  
  price: Number,  
  
  category: String,  
  
  imageUrl: String,  
  
});  
  
const User = mongoose.model('Product', ProductSchema);
```

User Schema

```
const UserSchema = new mongoose.Schema({  
  
  name: String,  
  
  email: String, age:  
  
  Number, address:  
  
  String,  
  
  password:String,  
  
  profileImage: String,  
  
});  
  
const User = mongoose.model('User', UserSchema);
```

Wishlist Item Schema

```
const WishlistItemSchema = new mongoose.Schema({
  userId: { type: String, required: true },
  productId: { type: mongoose.Schema.Types.ObjectId, ref:
    'Product' }, name: String,
  Brand: String,
  price: Number,
  quantity: Number,
  imageUrl: String,
});
const WishlistItem = mongoose.model("WishlistItem", WishlistItemSchema, "wishlist-items");
```

Cart Item Schema

```
const CartItemSchema = new mongoose.Schema({
  userId: { type: String, required: true },
  productId: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true },
  quantity: { type: Number, required: true, min: 1 },
  addedAt: { type: Date, default: Date.now }
});
const Cart = mongoose.model("CartItem", CartItemSchema, "cart-items");
```

3.6 MODULES DESCRIPTION

3.6.1 User Authentication Module

This module is responsible for user registration, login, and account management. New users can sign up by providing necessary credentials like name, email, phone number, and password, which are securely stored in the database. The login system ensures only authenticated users can access the app's core features. The module includes validation checks to prevent invalid entries and ensures secure password handling through hashing. This module forms the foundation for role-based access, enabling the app to distinguish between customers and admin users. It may also include password reset functionality and email/OTP verification in future versions for added security.

3.6.2 Product Management Module

This module allows admins to manage the inventory of steel products available in the store. Products can be added, updated, or deleted using an intuitive admin interface. Each product includes attributes such as name, category, dimensions, price, description, and stock quantity. The module ensures that any updates made by the admin are instantly reflected in the user interface. Backend APIs built using Node.js handle the CRUD (Create, Read, Update, Delete) operations efficiently and ensure real-time database synchronization. Images of the products can also be uploaded to improve user engagement. The data is stored and managed in MongoDB, allowing for flexible and scalable storage.

3.6.3 Product Browsing and Search Module

This is the customer-facing interface where users can browse through available steel products. Products are organized into categories for easier navigation. Users can view detailed product information including specifications, price, and availability. The search functionality allows users to quickly find specific products by name or category. This module ensures that users can make informed decisions by providing clear and detailed product descriptions. It fetches real-time product data from the backend and displays it using efficient React Native components for a smooth and responsive UI.

3.6.4 Cart and Order Placement Module

This module enables users to add selected products to their shopping cart and proceed to checkout. Users can adjust the quantity of items, remove products, and view the total cost before placing the order. The module calculates pricing dynamically and ensures product availability before checkout. Upon confirmation, the order data is passed to the backend for processing and saved in the database. The cart data is stored temporarily, ensuring users can pick up where they left off. This module also allows for previewing orders before final submission, improving user trust and reducing cart abandonment rates.

3.6.5 Payment Integration Module (Razorpay)

This module handles the payment process using Razorpay, one of India's leading payment gateways. After finalizing the cart, users are redirected to the Razorpay interface where they can choose various payment methods like UPI, credit/debit card, and net banking. The module verifies the transaction status and updates the order status accordingly. Successful payments trigger backend processes such as order confirmation and stock updates. Failed transactions are handled gracefully with user notifications and retry options. Razorpay's APIs are used to fetch payment details, generate payment links, and verify payment success callbacks. This integration enhances user trust and enables seamless transactions.

3.6.6 Admin Dashboard Module

This module provides backend control and analytics for the business owner or admin. It allows the admin to view and manage all aspects of the store, including user accounts, product listings, inventory levels, and order histories. Admins can add or remove products, update prices, monitor stock, and view recent transactions. The dashboard is built using simple yet powerful UI components and connects directly with backend APIs for real-time data management. It also includes notification mechanisms for low-stock alerts and pending orders. In the future, this module can be enhanced to include sales graphs, user behavior analytics, and order trends. It plays a critical role in business decision-making.

CHAPTER 4

TESTING

4.1 SYSTEM TESTING

System testing is a comprehensive evaluation conducted to ensure that the complete and integrated system meets the defined requirements. In the ARM Steel Agency e-commerce application, system testing encompassed core functionalities such as user registration, login authentication, product listing, cart management, order placement, and payment integration using Razorpay. Each module was rigorously tested under varied conditions to validate that all components interact seamlessly. Emphasis was placed on the end-to-end user flow, right from browsing products to completing a successful transaction. The testing ensured that UI responsiveness, backend communication, data consistency with MongoDB, and third-party service integration all worked harmoniously. The primary goal was to confirm that the application delivers a stable, intuitive, and error-free experience aligned with user expectations.

4.2 UNIT TESTING

Unit testing is the first level of testing and focuses on testing individual components or functions in isolation. In the ARM Steel Agency E-commerce App, unit tests were created for both frontend and backend elements. React Native components such as buttons, forms, input fields, and product display cards were tested to ensure they render correctly and respond appropriately to user actions like clicks and text inputs. The logic behind cart updates, quantity changes, and validations was verified using mock data. On the backend, Node.js functions responsible for user authentication, data validation, and MongoDB operations were tested. These tests helped catch logical errors, improper state updates, and unexpected responses early in the development cycle, ensuring code reliability and easier debugging.

4.3 MODULE TESTING

Module testing was conducted to assess the performance and correctness of grouped functionalities within the application. Each module was treated as a separate functional unit and tested for internal consistency. For instance, the User Module was tested for signup, login, session handling, and token generation; the Product Module was evaluated for adding, editing, and viewing products; the Cart Module was tested for managing cart items, and the Payment Module was checked for integrating Razorpay API and handling payment statuses. By focusing on each module individually, this testing ensured that internal logic, flow of control, and data processing were accurate, even before connecting the modules to others in the system.

4.4 INTEGRATION TESTING

Integration testing involved testing the interaction between different modules and services of the application. It validated whether modules like authentication, cart, and payment gateway communicate and function correctly when integrated. For example, after a user logs in, the session token should allow access to their cart, and subsequent actions like adding items and making payments should reflect accurately in the backend. API calls made from the React Native frontend to Express.js backend routes were tested to verify correct responses and error handling. Integration with third-party services like Razorpay and the MongoDB database was tested for scenarios such as payment success/failure and database connection issues. This ensured data consistency, proper error propagation, and seamless user flow across modules.

4.5 VALIDATION TESTING

Validation testing confirmed that the final software product met all defined functional and non-functional requirements. The application was validated against original business goals: smooth product browsing, secure user login, effective product search, real-time cart updates, reliable order management, and secure payment processing. It included field validation for form inputs, product quantity limits, and user authentication tokens.

Error scenarios were tested thoroughly, such as invalid login credentials, incorrect Razorpay keys, and loss of network connectivity during checkout. The app was also validated for UI responsiveness on multiple screen sizes. This phase ensured that the entire app functions correctly under realistic conditions, fulfilling both technical and user-centric goals.

CHAPTER 5

RESULTS AND DISCUSSION

The E-Commerce App developed for ARM Steel Agency has effectively delivered a comprehensive mobile solution for managing steel product sales and operations. It allows users to easily register, log in, explore a wide range of steel products with detailed specifications, add items to their cart, and securely complete purchases using Razorpay's integrated payment gateway. The backend, powered by Node.js and Express.js, works seamlessly with experience across both Android and iOS platforms. Admin functionalities enable efficient inventory control, order status updates, and transaction monitoring, reducing manual work and improving operational efficiency. Extensive testing confirmed smooth performance, quick response times, and reliable payment processing. The app enhances customer MongoDB to handle data operations like product listings, user management, and order tracking in real time. Built using React Native, the app offers a consistent and responsive user engagement through its clean interface and simple navigation, encouraging repeat usage. In terms of long-term value, the application is built with scalability in mind, allowing future enhancements like user analytics, multilingual support, and advanced reporting. Overall, the app represents a modern, digital-first approach to business, aligning with ARM Steel Agency's growth objectives and setting the foundation for continued expansion in the e-commerce domain.

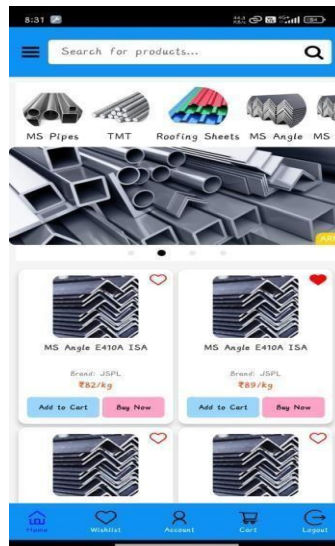


Figure 5.1 HOME SCREEN

The Home Screen is the main landing page of the system where users get an overview of the company and its purpose.

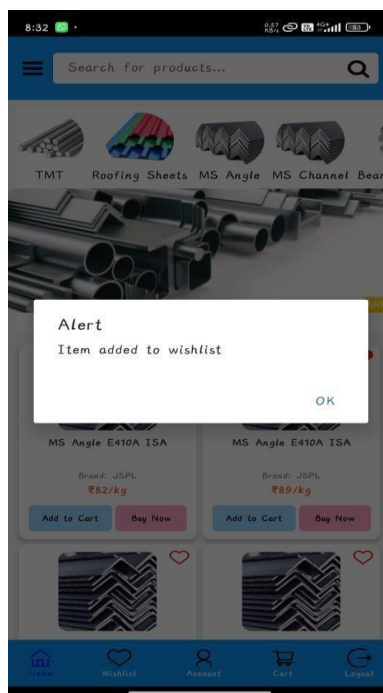


Figure 5.2 ITEM ADDED TO THE CART

This image illustrates that the item has been successfully added to your cart. You can continue shopping or proceed to checkout to complete your purchase.

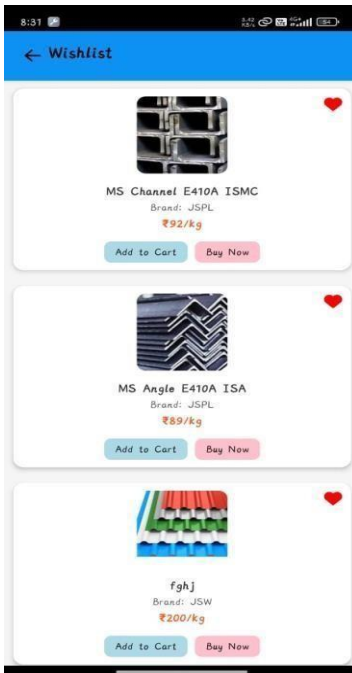


Figure 5.3 WISHLIST SCREEN

This image illustrates the items that has been added to the wishlist of the customer



Figure 5.4 PAYMENT SCREEN

The Payment Screen allows users to review their order details and choose a preferred payment method. It ensures a secure transaction process to complete the purchase successfully.

CHAPTER 6

CONCLUSION AND FUTURE WORK

The E-Commerce App developed for ARM Steel Agency has achieved its core purpose of creating a user-friendly, scalable, and efficient platform to manage the online sale of steel products. By incorporating modern technologies such as React Native for a cross-platform interface, Node.js and Express.js for server-side operations, and MongoDB for database management, the app ensures robust performance and seamless functionality. It simplifies the customer journey from product discovery to secure payment using Razorpay, while also providing the admin team with tools for effective inventory and order management.

The app's deployment has significantly improved business efficiency by reducing manual intervention, minimizing errors, and increasing customer accessibility to steel products. It bridges the gap between traditional sales methods and modern e-commerce, positioning ARM Steel Agency for digital growth and better market reach.

Looking toward the future, there is potential to incorporate additional features such as AI-based product recommendations, order tracking via GPS, integration with logistics providers, and advanced sales analytics dashboards. Multi-language support can cater to regional audiences, while implementing a rating and review system will help build trust and transparency. Furthermore, integrating real-time customer chat support and offering business-to-business (B2B) purchase options can expand the platform's usability. These enhancements will strengthen customer relationships, drive engagement, and support the long-term scalability and adaptability of the application in an evolving digital marketplace.

APPENDIX 1

CODING

App.js :

```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View } from 'react-native';
import RoofingSheetsScreen from './screens/categoryScreens/RoofingSheetsScreen'
import BuyNow from './screens/BuyNow';

const Stack=createNativeStackNavigator();

const Auth = () => {
  // Stack Navigator for Login and Sign up Screen
  return (
    <Stack.Navigator initialRouteName="Login">
      <Stack.Screen
        name="Login"
        component={Login}
        options={{headerShown: false}}
      />
    </Stack.Navigator>
  );
};

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName='Splash'>
        <Stack.Screen
          name="Splash"
          component={Splash}
          // Hiding header for Splash Screen
          options={{headerShown: false}}
        />
        { /* Auth Navigator: Include Login and Signup */ }
        <Stack.Screen
          name="Auth"
          component={Auth}
          options={{headerShown: false}}
        />
        <Stack.Screen
          name='home'
          component={Home}
          options={{
            headerShown:false,
          }}/>
        <Stack.Screen
          name='about'
```

```

        component={About}
        options={{
          headerShown:false,
        }}/>
        <Stack.Screen name='productDetails' options={{headerShown:false}}
component={ProductDetails}/>
        <Stack.Screen name="Wishlist" options={{headerShown:false}} component={Wishlist} />
        <Stack.Screen name="Cart" options={{headerShown:false}} component={Cart} />
        <Stack.Screen name="Profile" options={{headerShown:false}}
component={ProfileScreen} />
        <Stack.Screen name="roofing_sheets" component={RoofingSheetsScreen} />
        <Stack.Screen name="others" component={OthersScreen} />
        <Stack.Screen name="msflats" component={MSFlatsScreen} />
      </Stack.Navigator>
    </NavigationContainer>

  );
}

```

Product.js:

```

import { useNavigation } from "@react-navigation/native";
import AsyncStorage from '@react-native-async-storage/async-storage';
import { API_URL } from '@env';

```

```

const Products = ({ ListHeaderComponent }) => {
  const [products, setProducts] = useState([]);
  const [wishlistItems, setWishlistItems] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const navigation = useNavigation();
  const quantity=1;

  const handleProductPress = (id) => {
    navigation.navigate("productDetails", { _id: id });
  };

  if (!response.ok) {
    const errorData = await response.json();
    throw new Error(errorData.message || 'Failed to add item to cart');
  }
  alert("Item added to cart successfully!");
} catch (error) {
  console.error("Error adding to cart:", error);
  alert(`Failed to add item to cart: ${error.message}`);
}
}, []);

const handleAddToWishlist = async (productId) => {
  try {
    const userId = await AsyncStorage.getItem('user_id');
    if (!userId) {
      alert("Please login to add items to wishlist");
      return;
    }
  }
}

```

```

    if (!productId) {
      console.error("Product ID is missing");
      alert("Could not update wishlist: Product ID is missing");
      return;
    }

    const isInWishlist = wishlistItems.includes(productId);
    console.log("Operation:", isInWishlist ? "remove" : "add");
    console.log("User ID:", userId);
    console.log("Product ID:", productId);
    const handleBuyNow = (item) => {
      navigation.navigate('BuyNow', { item });
      console.log("Product id at products.js", item);
    };
    if (loading) {
      return (
        <View style={styles.centered}>
          <ActivityIndicator size="large" color="#ff5722" />
          <Text style={{ marginTop: 10 }}>Loading products...</Text>
        </View>
      );
    }
    if (error) {
      return (
        <View style={styles.centered}>
          <Text style={styles.errorText}>{error}</Text>
          <TouchableOpacity
            style={styles.retryButton}
            onPress={() => {
              setError(null);
              setLoading(true);
              fetch(`${API_URL}/products`)
                .then(response => response.json())
                .then(data => {
                  const productArray = Array.isArray(data) ? data : (data.products || data.data || []);
                  setProducts(productArray);
                  setLoading(false);
                })
                .catch(error => {
                  console.error("Error fetching products:", error);
                  setError("Failed to load products. Please try again later.");
                  setLoading(false);
                });
            }}
          >
          <Text style={styles.buttonText}>Retry</Text>
        </TouchableOpacity>
      </View>
    );
  }
  return (
    <FlatList

```

```

data={products}
keyExtractor={(item) => String(item._id)}
numColumns={2}
ListHeaderComponent={ListHeaderComponent}
contentContainerStyle={styles.listContainer}
renderItem={({ item }) => (
  <TouchableOpacity
    style={styles.productCard}
    onPress={() => handleProductPress(item._id)}
  >
    <TouchableOpacity
      style={styles.wishlistIcon}
      onPress={(e) => {
        e.stopPropagation();
        handleAddToWishlist(item._id);
      }}
    >
      <AntDesign
        name={wishlistItems.includes(item._id) ? "heart" : "hearto"}
        size={20}
        color="red"
      />
    </TouchableOpacity>
    <View style={styles.imageContainer}>
      <Image
        source={{ uri: item.ImageUrl }}
        style={styles.image}
        onError={(e) =>
          console.log("Image failed to load:", item.ImageUrl)
        }
      />
    </View>
    <Text style={styles.name} numberOfLines={4}>
      {item.name}
    </Text>
    <Text style={styles.brand}>Brand: {item.Brand}</Text>
    <Text style={styles.price}>₹ {item.price}/kg</Text>
    <View style={styles.buttonContainer}>
      <TouchableOpacity
        style={styles.addToCartButton}
        onPress={(e) => {
          e.stopPropagation();
          handleAddToCart(item._id);
        }}
      >
        <Text style={styles.buttonText}>Add to Cart</Text>
      </TouchableOpacity>
      <TouchableOpacity
        style={styles.buyNowButton}
        onPress={() => handleBuyNow({ item })}
      >
        <Text style={styles.buttonText}>Buy Now</Text>
      </TouchableOpacity>
    </View>
  </TouchableOpacity>
)
)

```

```

        </TouchableOpacity>
      </View>
    </TouchableOpacity>
  )}
/>
);
};

```

Cart.js:

```

import { View, Text, Image, StyleSheet, FlatList, TouchableOpacity, SafeAreaView, Alert,
ActivityIndicator } from 'react-native';
import React, { useState, useEffect, useContext } from 'react';
const Cart = ({ navigation, route }) => {
  const [cartItems, setCartItems] = useState([]);
  const [loading, setLoading] = useState(true);
  const localNavigation = useNavigation();
  useEffect(() => {
    const getUserId = async () => {
      try {
        const storedUserId = await AsyncStorage.getItem('user_id');
        if (storedUserId) {
          setUserId(storedUserId);
          fetchCartItems(storedUserId);
        } else {
          setError("User not authenticated. Please log in.");
          setLoading(false);
        }
      } catch (err) {
        console.error("Error retrieving user ID:", err);
        setError("Failed to authenticate. Please log in again.");
        setLoading(false);
      }
    };
    getUserId();
  }, []);
  const fetchCartItems = async (userIdParam) => {
    try {
      setLoading(true);
      const id = userIdParam || userId;
      if (!id) {
        throw new Error("User ID not available");
      }
      const response = await fetch(`${API_URL}/cart?userId=${id}`);
      if (!response.ok) {
        throw new Error(`HTTP error! Status: ${response.status}`);
      }
      const items = await response.json();
      setCartItems(items);
    } catch (err) {
      console.error("Error fetching cart items:", err);
      setError("Failed to load your cart items. Please try again.");
    }
  };

```

```

    } finally {
      setLoading(false);
    }
  };
const removeItemFromDatabase = async (itemId) => {
  try {
    setUpdatingItem(itemId);
    const response = await fetch(`${API_URL}/cart/${itemId}`, {
      method: 'DELETE',
    });
    if (!response.ok) {
      throw new Error(`Failed to remove item. Status: ${response.status}`);
    }
    setCartItems(prevItems => prevItems.filter(item => item._id !== itemId));
  } catch (err) {
    console.error("Error removing item:", err);
    Alert.alert("Remove Failed", "Failed to remove item. Please try again.");
    fetchCartItems(userId);
  } finally {
    setUpdatingItem(null);
  }
};
const increaseQuantity = (itemId, currentQuantity) => {
  const newQuantity = currentQuantity + 1;
  updateItemInDatabase(itemId, newQuantity);
};
const decreaseQuantity = (itemId, currentQuantity) => {
  if (currentQuantity > 1) {
    const newQuantity = currentQuantity - 1;
    updateItemInDatabase(itemId, newQuantity);
  }
};
const handleRemoveItem = (itemId) => {
  Alert.alert(
    "Remove Item",
    "Are you sure you want to remove this item from your cart?",
    [
      {
        text: "Cancel",
        style: "cancel"
      },
      {
        text: "Remove",
        onPress: () => removeItemFromDatabase(itemId),
        style: "destructive"
      }
    ]
  );
};
const calculateTotalPrice = () => {
  return cartItems.reduce((total, item) => total + (item.price * item.quantity), 0);
};

```

```

const handleCheckout = () => {
  if (cartItems.length === 0) {
    Alert.alert("Cart Empty", "Please add items to your cart before checkout.");
    return;
  }
  navigation.navigate('Checkout', { cartItems, totalAmount: calculateTotalPrice() });
};
const handleContinueShopping = () => {
  navigation.navigate('home');
};
const renderCartItem = ({ item }) => {
  const isUpdating = updatingItem === item._id;
  return (
    <View style={styles.cartItem}>
      {isUpdating && (
        <View style={styles.updatingOverlay}>
          <ActivityIndicator size="small" color="#ff5722" />
        </View>
      )}
      <View style={styles.itemImageContainer}>
        <Image
          source={{ uri: item.ImageUrl }}
          style={styles.itemImage}
          defaultSource={require('./assets/placeholder.png')}
          onError={() => console.log("Failed to load image:", item.ImageUrl)}
        />
      </View>
      <View style={styles.itemDetails}>
        <Text style={styles.itemName} numberOfLines={1}>{item.name}</Text>
        <Text style={styles.itemBrand}>{item.Brand || 'No Brand'}</Text>
        <Text style={styles.itemPrice}>₹ {item.price}/kg</Text>
        <View style={styles.quantityContainer}>
          <TouchableOpacity
            style={styles.quantityButton}
            onPress={() => decreaseQuantity(item._id, item.quantity)}
            disabled={isUpdating || item.quantity <= 1}
          >
            <Text style={[
              styles.quantityButtonText,
              (isUpdating || item.quantity <= 1) && styles.disabledText
            ]}>-</Text>
          </TouchableOpacity>
          <Text style={styles.itemQuantity}>{item.quantity}</Text>
          </TouchableOpacity>
        </View>
      </View>
      <View style={styles.itemActions}>
        <Text style={styles.itemTotalPrice}>₹ {(item.price * item.quantity).toFixed(2)}</Text>
        <TouchableOpacity
          style={styles.removeButton}
          onPress={() => handleRemoveItem(item._id)}
          disabled={isUpdating}
        >

```

```

      <AntDesign name="delete" size={20} color={isUpdating ? "#ccc" : "#ff5722"} />
    </TouchableOpacity>
  </View>
</View>
);
};

const renderEmptyCart = () => {
  return (
    <View style={styles.emptyCartContainer}>
      <AntDesign name="shoppingcart" size={80} color="#ccc" />
      <Text style={styles.emptyCartText}>Your cart is empty</Text>
      <TouchableOpacity
        style={styles.continueShopping}
        onPress={handleContinueShopping}
      >
        <Text style={styles.continueShoppingText}>Continue Shopping</Text>
      </TouchableOpacity>
    </View>
  );
};

if (loading) {
  return (
    <View style={styles.centered}>
      <ActivityIndicator size="large" color="#ff5722" />
      <Text style={{ marginTop: 10 }}>Loading your cart...</Text>
    </View>
  );
}

if (error) {
  return (
    <View style={styles.centered}>
      <Text style={styles.errorText}>{error}</Text>
      <TouchableOpacity
        style={[styles.button, { marginTop: 20, backgroundColor: '#ff5722' }]}
        onPress={fetchCartItems}
      >
        <Text style={[styles.buttonText, { color: 'white' }]}>Try Again</Text>
      </TouchableOpacity>
    </View>
  );
}

return (
  <SafeAreaView style={styles.container}>
    <View style={styles.header}>
      <TouchableOpacity
        style={styles.backButton}
        onPress={() => navigation.goBack()}

```

```

    >
    <AntDesign name="arrowleft" size={24} color="black" />
  </TouchableOpacity>
  <Text style={styles.headerTitle}>Shopping Cart</Text>
  <View style={styles.placeholder}></View>
</View>

{cartItems.length > 0 ? (
  <
    <FlatList
      data={cartItems}
      renderItem={renderCartItem}
      keyExtractor={item => item._id}
      contentContainerStyle={styles.cartList}
      showsVerticalScrollIndicator={false}
    />

    <View style={styles.cartSummary}>
      <View style={[styles.summaryRow, styles.totalRow]}>
        <Text style={styles.totalLabel}>Total:</Text>
        <Text style={styles.totalValue}>₹ {(calculateTotalPrice()).toFixed(2)}</Text>
      </View>

      <TouchableOpacity
        style={styles.checkoutButton}
        onPress={handleCheckout}
      >
        <Text style={styles.checkoutText}>Proceed to Checkout</Text>
      </TouchableOpacity>
    </View>
  </>
) : (
  renderEmptyCart()
)}
</SafeAreaView>
);
};

```

Checkout.js :

```

import React, { useState } from 'react';
import { View, Text, TextInput, Button, StyleSheet, Alert } from 'react-native';

export default function Checkout({ route, navigation }) {
  const { cartItems, totalAmount } = route.params;
  console.log("Total price:", totalAmount);
  const [address, setAddress] = useState("");
  const [distance, setDistance] = useState("");

  const transportFee = parseInt(distance || 0) * 80;
  const grandTotal = totalAmount + transportFee;
  const handleNext = () => {

```

```

    if(!address || !distance) {
      Alert.alert('Missing Info', 'Please enter address and distance.');
```

```

    return;
  }
  navigation.navigate('Payment', {
    cartItems,
    grandTotal,
    address,
    transportFee,
    distance,
  });
};

return (
  <View style={styles.container}>
    <Text style={styles.title}>Shipping Address</Text>

    <TextInput
      placeholder="Enter shipping address"
      style={styles.input}
      value={address}
      onChangeText={setAddress}
    />

    <TextInput
      placeholder="Distance from shop (in km)"
      style={styles.input}
      keyboardType="numeric"
      value={distance}
      onChangeText={setDistance}
    />

    <Text style={styles.fee}>Transport Fee: ₹{transportFee}</Text>
    <Text style={styles.total}>Grand Total: ₹{grandTotal}</Text>

    <Button title="Proceed to Payment" onPress={handleNext} disabled={!address || !distance} />
  </View>
);
}

```

Home.js :

```

import React from 'react'
import { StyleSheet } from 'react-native'
import Layout from '../components/Layout/Layout'
import Products from '../components/Products/Products'
import Header from '../components/Layout/Header'
import Categories from '../components/category/Categories'
import Banner from '../components/Banner/Banner'
import { View } from 'react-native'

const Home = () => {
  const ListHeader = () => (

```

```

    <View style={styles.headerContainer}>
      <Categories />
      <Banner />
    </View>
  )

  return (
    <Layout style={styles.container}>
      <Header />
      <Products ListHeaderComponent={ListHeader} />
    </Layout>
  )
}

```

export default Home

LoginScreen.js :

```

import React, {useState, createRef} from 'react';
import {
  StyleSheet,
  TextInput,
  View,
  Text,
  ScrollView,
  Image,
  Keyboard,
  TouchableOpacity,
  KeyboardAvoidingView,
} from 'react-native';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { API_URL } from '@env';

const LoginScreen = ({navigation}) => {
  const [userEmail, setUserEmail] = useState("");
  const [userPassword, setUserPassword] = useState("");
  const [loading, setLoading] = useState(false);
  const [errortext, setErrortext] = useState("");

  const passwordInputRef = createRef();

  const handleSubmitPress = () => {
    setErrortext("");
    if (!userEmail) {
      alert('Please fill Email');
      return;
    }
    if (!userPassword) {
      alert('Please fill Password');
      return;
    }
    setLoading(true);
  }

```

```

let dataToSend = {email: userEmail, password: userPassword};
let formBody = [];
for (let key in dataToSend) {
  let encodedKey = encodeURIComponent(key);
  let encodedValue = encodeURIComponent(dataToSend[key]);
  formBody.push(encodedKey + '=' + encodedValue);
}
formBody = formBody.join('&');

console.log("API_URL is: ", API_URL);

return (
  <View style={styles.mainBody}>
    <ScrollView
      keyboardShouldPersistTaps="handled"
      contentContainerStyle={{
        flex: 1,
        justifyContent: 'center',
        alignContent: 'center',
      }}>
      <View>
        <KeyboardAvoidingView enabled>
          <View style={{alignItems: 'center'}}>
            <Image
              source={require('./image/logo-arm.png')}
              style={{
                width: '50%',
                height: 100,
                resizeMode: 'contain',
                margin: 30,
                borderRadius: 50
              }}
            />
          </View>
          <View style={styles.SectionStyle}>
            <TextInput
              style={styles.inputStyle}
              onChangeText={(UserEmail) =>
                setUserEmail(UserEmail)
              }
              placeholder="Enter Email" //dummy@abc.com
              placeholderTextColor="#8b9cb5"
              autoCapitalize="none"
              keyboardType="email-address"
              returnKeyType="next"
              onSubmitEditing={() =>
                passwordInputRef.current &&
                passwordInputRef.current.focus()
              }
            />
          </View>
        </KeyboardAvoidingView>
      </View>
    </ScrollView>
  </View>
);

```

```

        underlineColorAndroid="#f000"
        blurOnSubmit={false}
      />
    </View>
    <View style={styles.SectionStyle}>
    </View>
    <TouchableOpacity
      style={styles.buttonStyle}
      activeOpacity={0.5}
      onPress={handleSubmitPress}>
      <Text style={styles.buttonTextStyle}>LOGIN</Text>
    </TouchableOpacity>
    <Text
      style={styles.registerTextStyle}
      onPress={() => navigation.navigate('Register')}>
      New Here ? Register
    </Text>
  </KeyboardAvoidingView>
</View>
</ScrollView>
</View>
);
};
export default LoginScreen;

```

ProductDetails.js :

```

import {
  View,
  Text,
  Image,
  StyleSheet,
  ScrollView,
  TouchableOpacity,
  ActivityIndicator,
} from "react-native";
import React, { useEffect, useState } from "react";
import { AntDesign } from "@expo/vector-icons";
import AsyncStorage from "@react-native-async-storage/async-storage";
import { API_URL } from "@env";

const ProductDetails = ({ route, navigation }) => {
  const [products, setProducts] = useState([]);
  const [product, setProduct] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [quantity, setQuantity] = useState(1);
  const [wishlistItems, setWishlistItems] = useState([]);

  // Safely extract the product ID from route params
  const productId = route?.params?._id;
  useEffect(() => {

```

```

    if (!productId) {
      setError("No product ID provided");
      setLoading(false);
      return;
    }
    useEffect(() => {
      const loadWishlist = async () => {
        try {
          const userId = await AsyncStorage.getItem("user_id");
          if (!userId) return;

          const response = await fetch(`${API_URL}/wishlist?userId=${userId}`);
          if (!response.ok) {
            throw new Error("Failed to fetch wishlist");
          }

          const data = await response.json();
          // Extract product IDs from wishlist items
          const wishlistProductIds = data.map((item) => item.productId);
          console.log("Loaded wishlist items:", wishlistProductIds.length);
          setWishlistItems(wishlistProductIds);
        } catch (error) {
          console.error("Error loading wishlist:", error);
        }
      };

      loadWishlist();
    }, []);
    const handleAddToCart = async (productId, quantity) => {
      try {
        const userId = await AsyncStorage.getItem("user_id");

        if (!userId) {
          alert("Please login to add items to cart");
          return;
        }
        const response = await fetch(`${API_URL}/add-to-cart`, {
          method: "POST",
          headers: {
            "Content-Type": "application/json",
          },
          body: JSON.stringify({
            userId,
            productId,
            quantity,
          }),
        });
      };

      const handleAddToWishlist = async (productId) => {
        try {
          const userId = await AsyncStorage.getItem("user_id");

```

```

    if (!userId) {
      alert("Please login to add items to wishlist");
      return;
    }
    const isInWishlist = wishlistItems.includes(productId);
    console.log("Operation:", isInWishlist ? "remove" : "add");
    console.log("User ID:", userId);
    console.log("Product ID:", productId);
    if (loading) {
      return (
        <View style={styles.centered}>
          <ActivityIndicator size="large" color="#ff5722" />
          <Text style={{ marginTop: 10 }}>Loading product details...</Text>
        </View>
      );
    }

    return (
      <ScrollView style={styles.container}>
        { /* Product Image */ }
        <View style={styles.imageContainer}>
          <Image
            source={{ uri: product.imageUrl }}
            style={styles.image}
            onError={() => console.log("Failed to load image:", product.imageUrl)}
          />
          <TouchableOpacity
            style={styles.backButton}
            onPress={() => navigation.goBack()}
          >
            <AntDesign name="arrowleft" size={24} color="black" />
          </TouchableOpacity>
          <TouchableOpacity
            style={styles.wishlistIcon}
            onPress={(e) => {
              e.stopPropagation();
              handleAddToWishlist(product._id);
            }}
          >
            <AntDesign
              name={wishlistItems.includes(product._id) ? "heart" : "hearto"}
              size={24}
              color="red"
            />
          </TouchableOpacity>
        </View>
        { /* Description */ }
        <View style={styles.section}>
          <Text style={styles.sectionTitle}>Description</Text>
          <Text style={styles.description}>
            {product.description ||
              "No description available for this product. This premium quality product is carefully"}
          </Text>
        </View>
      </ScrollView>
    );
  }
}

```

```

selected to ensure you get the best value for your money."}
    </Text>
  </View>
  </TouchableOpacity>
</View>
</View>
</ScrollView>
);
};

```

RegisterScreen.js :

```

import React, {useState, createRef} from 'react';
import Loader from '../components/loader';
import { API_URL } from '@env';
const RegisterScreen = (props) => {

  var formBody = [];
  for (var key in dataToSend) {
    var encodedKey = encodeURIComponent(key);
    var encodedValue = encodeURIComponent(dataToSend[key]);
    formBody.push(encodedKey + '=' + encodedValue);
  }
  formBody = formBody.join('&');
  console.log(API_URL);
  fetch(`${API_URL}/api/user/register`, {
    method: 'POST',
    body: formBody,
    headers: {
      //Header Defination
      'Content-Type':
        'application/x-www-form-urlencoded;charset=UTF-8',
    },
  })
  .then((response) => response.json())
  .then((responseJson) => {
    setLoading(false);
    console.log(responseJson);
    if (responseJson.status === 'success') {
      setIsRegistrationSuccess(true);
      console.log(
        'Registration Successful. Please Login to proceed'
      );
    } else {
      setErrorText(responseJson.msg);
    }
  })
  .catch((error) => {
    setLoading(false);
    console.error(error);
  });
};

```

```

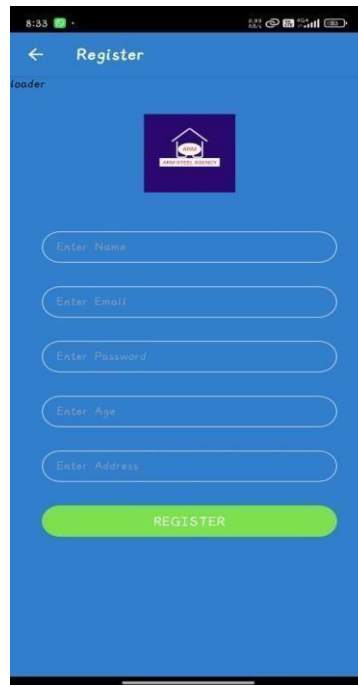
return (
  <View style={{flex: 1, backgroundColor: '#307ecc'}}>
    <Loader loading={loading} />
    <ScrollView
      keyboardShouldPersistTaps="handled"
      contentContainerStyle={{
        justifyContent: 'center',
        alignContent: 'center',
      }}>
      <KeyboardAvoidingView enabled>
        <View style={styles.SectionStyle}>
          <TextInput
            style={styles.inputStyle}
            onChangeText={({UserName}) => setUsername(UserName)}
            underlineColorAndroid="#f000"
            placeholder="Enter Name"
            placeholderTextColor="#8b9cb5"
            autoCapitalize="sentences"
            returnKeyType="next"
            onSubmitEditing={() =>
              emailInputRef.current && emailInputRef.current.focus()
            }
            blurOnSubmit={false}
          />
        </View>
        <View style={styles.SectionStyle}>
        </View>
        <View style={styles.SectionStyle}>
          <TextInput
            style={styles.inputStyle}
            onChangeText={({UserPassword}) =>
              setUserPassword(UserPassword)
            }
            underlineColorAndroid="#f000"
            placeholder="Enter Password"
            placeholderTextColor="#8b9cb5"
            ref={passwordInputRef}
            returnKeyType="next"
            secureTextEntry={true}
            onSubmitEditing={() =>
              ageInputRef.current &&
              ageInputRef.current.focus()
            }
            blurOnSubmit={false}
          />
        </View>
        </View>
        {errortext != " ? (
          <Text style={styles.errorTextStyle}>
            {errortext}
          </Text>
        ) : null}

```

```
    <TouchableOpacity
      style={styles.buttonStyle}
      activeOpacity={0.5}
      onPress={handleSubmitButton}>
      <Text style={styles.buttonTextStyle}>REGISTER</Text>
    </TouchableOpacity>
  </KeyboardAvoidingView>
</ScrollView>
</View>
);
};
export default RegisterScreen;
```

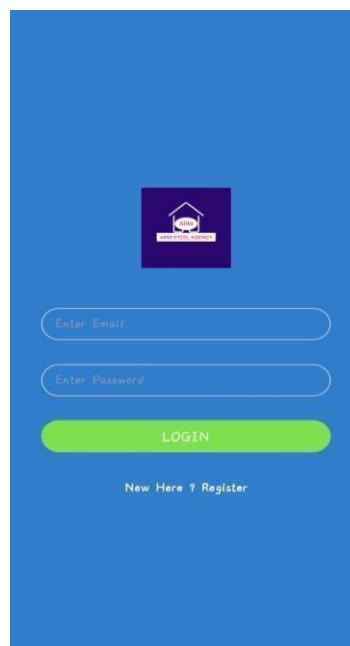
APPENDIX 2

SNAPSHOTS



A mobile application registration screen with a blue background. At the top, there is a status bar showing the time 8:33 and various icons. Below the status bar is a navigation bar with a back arrow and the title "Register". The main content area features a logo at the top center, which consists of a house icon with a speech bubble inside, containing the text "WELCOME HOME". Below the logo are five input fields with placeholder text: "Enter Name", "Enter Email", "Enter Password", "Enter Age", and "Enter Address". At the bottom of the form is a large green button labeled "REGISTER".

Fig. A2.1 Signup Screen



A mobile application login screen with a blue background. At the top, there is a logo at the top center, which consists of a house icon with a speech bubble inside, containing the text "WELCOME HOME". Below the logo are two input fields with placeholder text: "Enter Email" and "Enter Password". At the bottom of the form is a large green button labeled "LOGIN". Below the login button is a link that says "New Here ? Register".

Fig. A2.2 Login Screen

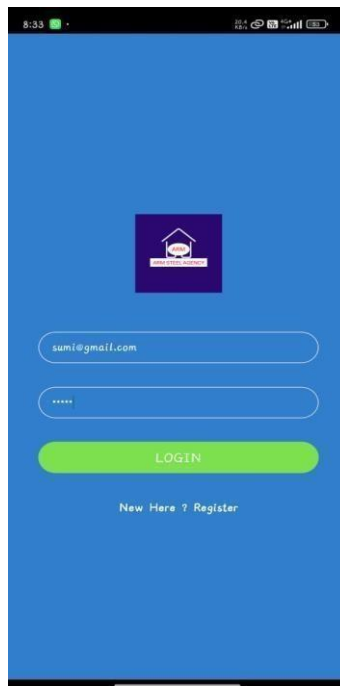


Fig. A2.3 Login Input Screen

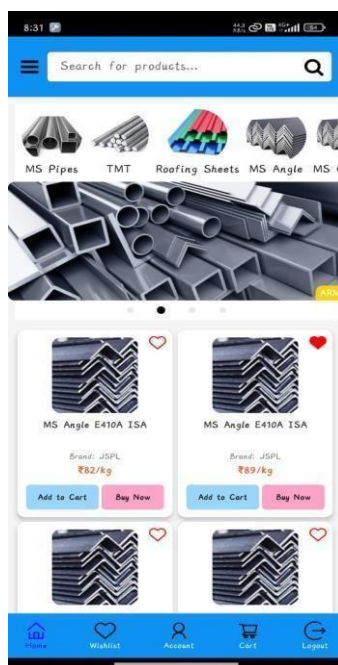


Fig. A2.4 Home Screen

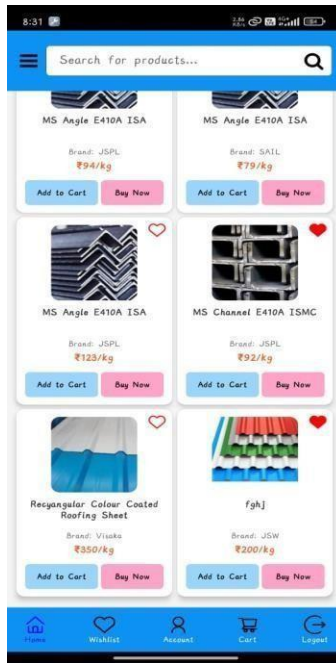


Fig. A2.5 Home Screen Scrolled

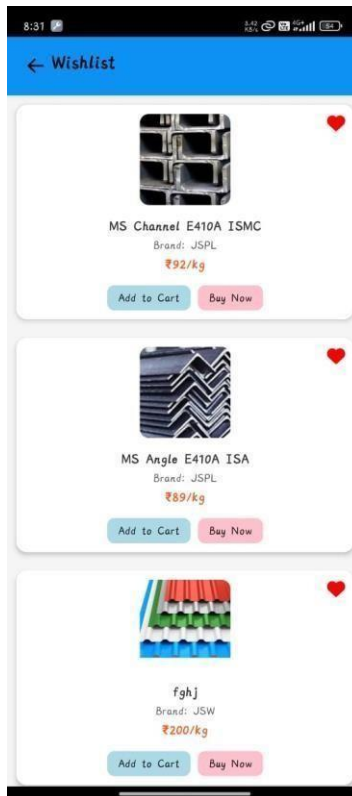


Fig. A2.6 Wishlist Screen

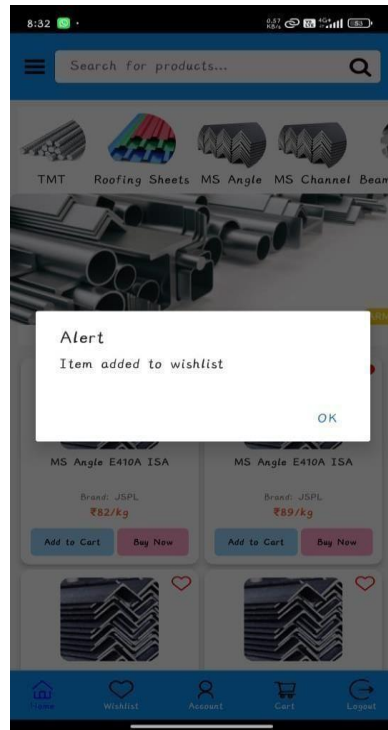


Fig. A2.7 Adding Item to Wishlist

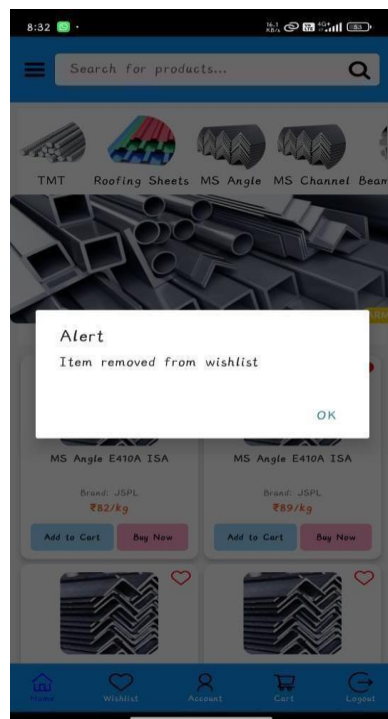


Fig. A2.8 Removing Item from Wishlist

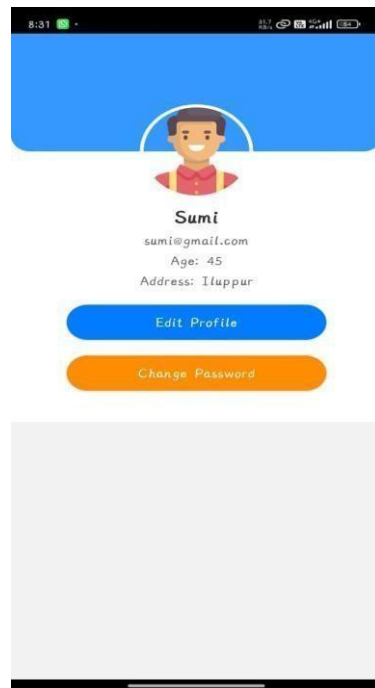


Fig. A2.9 Profile Screen

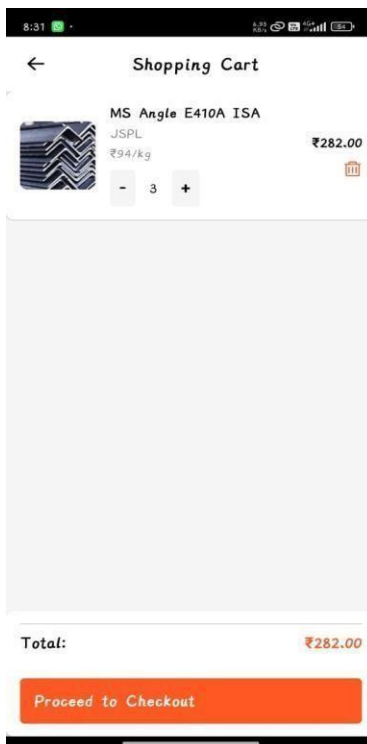


Fig. A2.10 Cart Screen

8:31

← Checkout

Shipping Address

Enter shipping address

Distance from shop (in km)

Transport Fee: ₹0

Grand Total: ₹282

PROCEED TO PAYMENT

This screenshot shows the checkout screen at 8:31. It features a 'Shipping Address' section with two input fields: 'Enter shipping address' and 'Distance from shop (in km)'. Below these fields, the 'Transport Fee' is listed as ₹0, and the 'Grand Total' is ₹282. A grey button labeled 'PROCEED TO PAYMENT' is positioned at the bottom of the form area.

Fig. A2.11 Checkout Screen

8:32

← Checkout

Shipping Address

143/2, Palani Main Road, Madathukulam

2

Transport Fee: ₹160

Grand Total: ₹442

PROCEED TO PAYMENT

This screenshot shows the checkout screen at 8:32. The 'Shipping Address' fields are now filled with the text '143/2, Palani Main Road, Madathukulam' and '2'. The 'Transport Fee' has increased to ₹160, and the 'Grand Total' is now ₹442. The 'PROCEED TO PAYMENT' button is now blue.

Fig. A2.12 Checkout Details



Fig. A2.13 Initializing Payment

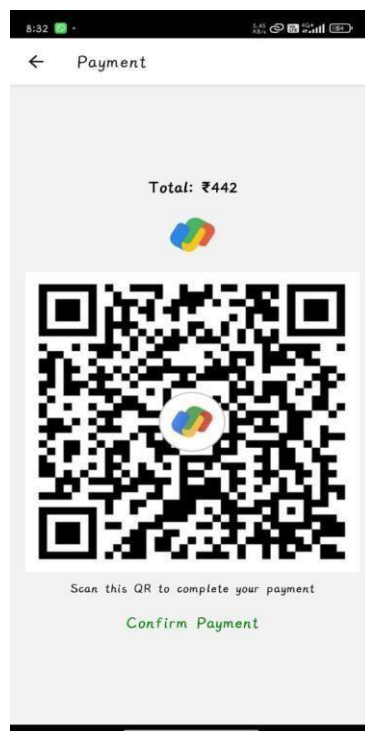


Fig. A2.14 Payment Screen

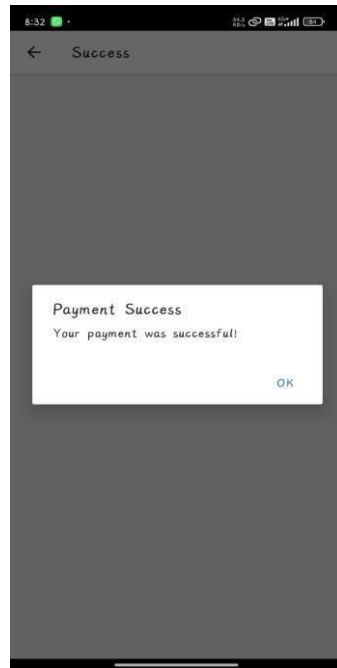


Fig. A2.15 Payment Confirmation

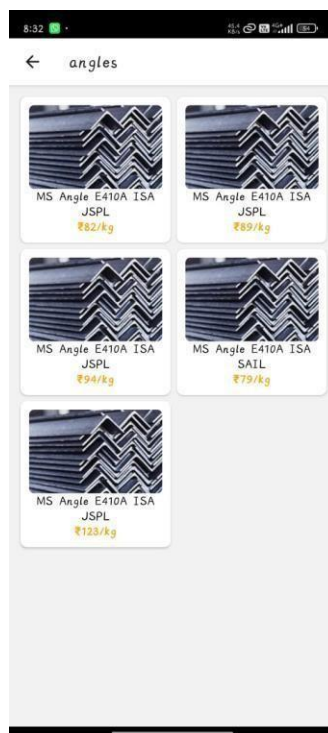


Fig. A2.16 Filter based on Categories

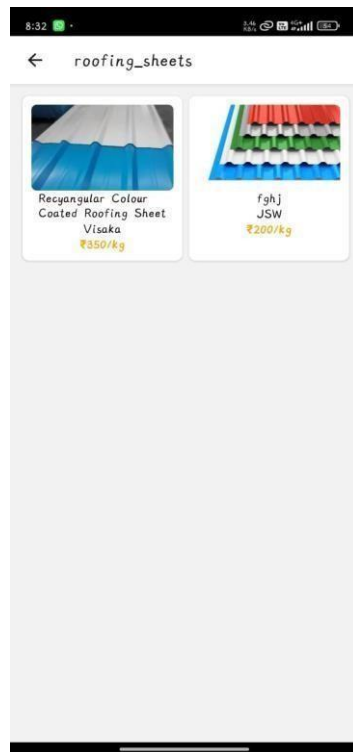


Fig. A2.17 Categories Screen – Roofing Sheets

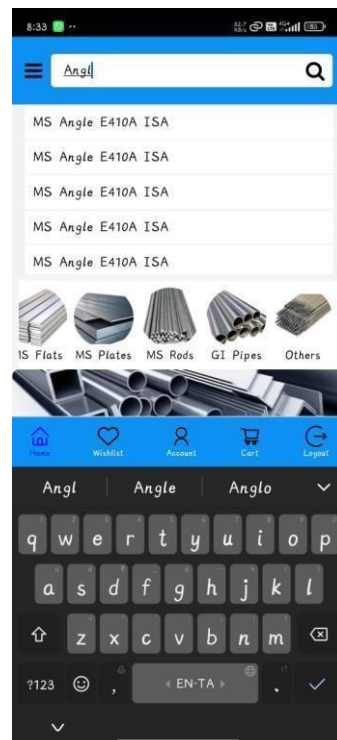


Fig. A2.18 Search Products

REFERENCES

1. **React Native Documentation** -Meta (Facebook), <https://reactnative.dev/docs/>
2. **Node.js Documentation** – OpenJS Foundation, <https://nodejs.org/en/docs>
3. **Express.js Guide** – Express.js Official Site, <https://expressjs.com/>
4. **MongoDB Manual** – MongoDB Inc., <https://www.mongodb.com/docs/manual/>
5. **Razorpay Integration Docs** - Razorpay Developer Portal,<https://razorpay.com/docs/>
6. <https://github.com/aravindbalajic/>
7. <https://github.com/DeepikaRajasundar/>
8. <https://github.com/dharsinijagadeesan/>

APPROVAL LETTER FROM THE COMPANY

GSTIN : 33AUFPT2134F12W 90428 54888
Off : 90428 63888

JAGAN WOOD FURNITURES
604/2, Kaliyampudur Private, Kunnathur Road, PERUNDURAI - 638 052, Erode Dt.
E-mail : jagantharini@gmail.com

Ref _____ Date : 09.02.2024

Dear Sir,

Sub: Industry Consultancy Project-reg.,

With reference to the discussion regarding the consultancy , we are ready to provide our industry project to IT department, **KONGU ENGINEERING COLLEGE, PERUNDURAI, ERODE.**

In this regard, we are pleased to select **NIHASINI P S (21ITR072), SOBIYA B (21ITR104) and SASI PRIYANKA J (21ITR097)** of IT Department for this project.

The details are as follows:

Title: E-commerce website for **JAGAN WOOD FURNITURES.**

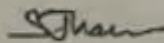
Duration: Three months.

The project is jointly monitored by the faculty guide, **Dr.E.M.Roopna Devi (Associate Professor)** of IT department. Upon completion of the project, the final report is to be submitted to both the company and the department.

In recognition of the students' s efforts a consultancy amount of Rs.10,030/-(including GST) is allocated for this project. We look forward to a successful collaboration.

Thanking you

For **JAGAN WOOD FURNITURES**


Proprietor

COMPLETION LETTER FROM THE COMPANY

GSTIN : 33AUFPT2134F1ZW 98428 54888
OFF : 98428 53888

JAGAN WOOD FURNITURES
554/2, Kaliyampudur Pirivu, Kunnathur Road, PERUNDURAI - 638 052, Erode Dt.
E-mail : jaganharin@gmail.com

Ref: _____ Date : 30-03-2024

Dear Sir/Madam,

We have priviledge in announce that the students, Nihasini P S (211TR072), Sasi Priyanka J (211TR097), Sobiya B (211TR104). Under the guidance Ms.E.M.Roopa Devi Associate Professor from the department of Information Technology, Kongu Engineering College, Perundurai have successfully completed development of the customized online shopping for our concern "Jagan Wood Furnitures" with the requirements:

- 1.Home Page
- 2.Product details with images
- 3.Admin Dashboard.
- 4.Payment Page
- 5.Cart

We are really satisfied with your software. We also request you to extend your support in future.

Thanking you

For JAGAN WOOD FURNITURES

RECEIPT

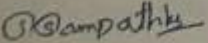
sri sarvam 

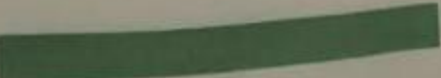
Sri Sarvam Educational Building,
Kannur, Malappuram,
Kerala - 676001
Telephone : 9788718180

Date : 06/05/2023

Receipt

K.DHANUSH(201TR019) , M.DHARANI(201TR021) ,
S.KARTHIKEYAN(201TL118) has completed the project of 'WEB APPLICATION
FOR SRI SARVAM OILS' in the cost of (10,000+GST 18%) 


Managing Director

☎ 9788718180 ✉ srisarvamoils@gmail.com 

MAPPING OF SDG TO PROJECT WORK – I

Sustainable Development Goals (SDGs) are a set of global goals established by the United Nations to promote sustainable development and improve the quality of life for people around the world. They focus on important areas such as economic growth, environmental protection, innovation, education, and social equality. By integrating sustainable practices and technologies, different sectors—including digital platforms like e-commerce—can contribute to achieving these goals and creating a better and more sustainable future for society.

SDG 9 – Industry, Innovation and Infrastructure

This project aligns with **SDG Goal 9** by introducing digital innovation into the traditional steel retail industry. **The ARM Steel Agency e-commerce** mobile application modernizes the steel purchasing process by enabling customers to browse products, place orders, and make secure payments online through Razorpay. By transforming a conventional retail business into a technology-driven platform, the project promotes industrial innovation and improves infrastructure for efficient trade and commerce.

Additionally, the system strengthens business operations through features such as real-time inventory management, order tracking, and an admin dashboard for monitoring sales and customer activities. These digital tools enhance operational efficiency, reduce manual processes, and support scalable growth. By integrating modern digital infrastructure into steel retail operations, the project contributes to building resilient industry systems and encourages technological adoption in traditional sectors, which is a core objective of **Sustainable Development Goal 9**.